

Metrics

May 20, 2026

1 Evaluation Metrics

We evaluated the LLM-generated code along four major software code quality dimensions: functional correctness, quality, maintainability, and security. The idea is to capture both technical accuracy and practical software engineering relevance. All metrics were computed automatically using established open-source tools, and their variance across influence tactic conditions was later analyzed using mixed-effects statistical models.

Functional Correctness: Functional correctness measures whether the generated code produces the outputs expected by the benchmark’s specific test cases. This is assessed using each dataset’s official evaluation harness. For each problem instance i , correctness $C_i \in \{0, 1\}$ where:

$$C_i = \begin{cases} 1 & \text{if all tests pass} \\ 0 & \text{otherwise} \end{cases}$$

Code Quality and Maintainability Metrics: To evaluate non-functional quality, we employed static-analysis measures commonly used in software engineering research: Cyclomatic Complexity (CC) [7], Maintainability Index (MI) [8], and PyLint [5]. We also report Source Lines of Code (SLOC) and percentage of comments (calculated as $\frac{\text{comments}}{\text{SLOC}} * 100\%$), to quantify verbosity and documentation density.

Metrics are useful because they provide automatically computable, reproducible signals that can help estimate different aspects of code. Chowdhury et al. show that code metrics improved prediction of change-proneness across 730K Java methods from 47 open-source projects, even after controlling for method size [4]. Their findings suggest that metrics can provide meaningful information beyond simple code size alone, particularly for identifying code that may be more likely to require future modification. Practitioners also perceive code complexity as negatively influencing readability, understandability, modifiability, and maintenance time [1].

These measures are therefore used as reproducible indicators for relative comparison across prompt conditions, rather than as complete substitutes for professional software evaluation. This distinction is important because prior work has shown that metric-based assessments may diverge from developers’

perceived quality improvements [9]. Similarly, developer-centered studies emphasize that important code quality properties, such as readability, structure, comprehensibility, and maintainability, are difficult to fully capture through static metrics alone [2]. Accordingly, the metrics in this study should be interpreted as structured quantitative signals that support comparison, not as exhaustive assessments of code quality in practice.

Cyclomatic complexity (CC), as proposed by McCabe [7], is a measure of the number of linearly independent paths in a program. This was calculated using the Radon [6] package for Python. We report both average cyclomatic complexity CC_{avg} , representing the mean complexity across all functions and methods in the generated code, and maximum cyclomatic complexity CC_{max} , which identifies the single most complex function or method, as it often represents the primary maintenance bottleneck.

Maintainability index (MI) is a metric used to assess how easily software can be maintained and evolved [8]. It aggregates structural characteristics (lines of code, complexity, and comment density) into a single interpretable score where higher values denote easier maintainability. We calculate MI using the Radon [6] Python package.

PyLint [5] is an overall assessment of a file’s errors and adherence to code standards. For LiveCodeBench, this was calculated with the PyLint library with default parameters. Conversely, for SWE-bench Verified, in order to avoid complications due to failing imports from other files potentially missing as context during evaluation, we disabled a number of import-related flags checked by PyLint (import-error, no-name-in-module, wrong-import-position, ungrouped-imports).

Security: We evaluated the security of LLM-generated code using Bandit [10], a static analysis tool that detects common Python vulnerabilities and classifies them as low, medium, or high severity. For each generated solution, we followed an approach similar to Chen & Jiang (2025) [3], recording the total counts and differences between post- and pre-patch vulnerabilities to estimate how prompt framings influenced vulnerability frequency. The usage of differences allows us to isolate the effect of the model’s edits by comparing the differences between the pre-patch and post-patch files.

Multi-File Metric Aggregation for SWE-bench Verified: The above metrics were computed on the full extracted code block for LiveCodeBench. Because SWE-bench patches can span multiple files, we measured metric differences (Δ) between the pre- and post-patch code to isolate changes attributable to the generated edits (similar to Chen & Jiang (2025) [3]’s approach). Cyclomatic complexity was aggregated across files by pooling the per-block values (e.g., for functions, methods, or classes) from all modified Python files into a unified list. The average (CC_{avg}) was then computed as the mean of this pooled list, while the maximum (CC_{max}) represents the highest value across all blocks. We then derived the differences as

$$\Delta CC_{avg} = \frac{1}{|B_{post}|} \sum_{b \in B_{post}} CC_b - \frac{1}{|B_{pre}|} \sum_{b \in B_{pre}} CC_b$$

$$\Delta CC_{max} = \max_{b \in B_{post}} CC_b - \max_{b \in B_{pre}} CC_b$$

where B represents all code blocks (functions, methods, classes) across modified files.

For MI, we computed the average maintainability across all the files, then the delta is derived as

$$\Delta MI = MI_{avg}^{post} - MI_{avg}^{pre}$$

For PyLint we computed the code for all files before the edits at once (as PyLint supports it), and then after the edits, and $\Delta PyLint$ is calculated by subtracting the two.

$$\Delta PyLint = PyLint_{post} - PyLint_{pre}$$

$\Delta Bandit$ is computed for all of high, medium, and low severity issues by summing the occurrences of high, medium, and low severity issues on the pre-patch files and post-patch files, then subtracting the two:

$$\Delta Bandit_s = \sum_{f \in F} |Issues_s^{post}(f)| - \sum_{f \in F} |Issues_s^{pre}(f)|$$

for each severity level $s \in \{high, medium, low\}$.

▲ Metric limitations: These metrics serve as interpretable proxies rather than exhaustive measures of software quality. MI and CC emphasize structural maintainability risks; PyLint captures style and static code-quality conventions; SLOC and comment percentage capture verbosity and documentation; Bandit detects only rule-based security vulnerabilities. No single metric is fully aligned with a professional developer’s assessment of whether a snippet is maintainable or high-quality in a production context. Therefore, we interpret results from each metric as relative comparisons across prompt conditions under the same tasks, models, and evaluation procedure. To partially address these limitations, we complement the quantitative metrics with a qualitative code-book analysis that captures higher-level stylistic and behavioural patterns in the generated code. This analysis does not replace developer judgment, but it provides an additional perspective on response characteristics that static metrics may miss, including communication style, tone, response structure, presence of code explanations, readability, commenting quality, error handling, and hallucination patterns.

References

- [1] Antinyan, V., Staron, M., Sandberg, A.: Evaluating code complexity triggers, use of complexity measures and the influence of code complexity on maintenance time. *Empirical Software Engineering* **22**(6), 3057–3087 (2017)
- [2] Börstler, J., Bennin, K.E., Hooshangi, S., Jeuring, J., Keuning, H., Kleiner, C., MacKellar, B., Duran, R., Störrle, H., Toll, D., et al.: Developers

- talking about code quality. *Empirical Software Engineering* **28**(6), 128 (2023)
- [3] Chen, Z., Jiang, L.: Evaluating software development agents: Patch patterns, code quality, and issue complexity in real-world github scenarios. In: 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 657–668. IEEE (2025)
 - [4] Chowdhury, S., Holmes, R., Zaidman, A., Kazman, R.: Revisiting the debate: Are code metrics useful for measuring maintenance effort? *Empirical Software Engineering* **27**(6), 158 (2022)
 - [5] contributors, P.: Pylint (2025). URL <https://github.com/pylint-dev/pylint>. Online; accessed 10 Jul 2025
 - [6] Lacchia, M.: Radon (2025). URL <https://github.com/rubik/radon>. Online; accessed 10 Jul 2025
 - [7] McCabe, T.J.: A complexity measure. *IEEE Transactions on software Engineering* **SE-2**(4), 308–320 (1976)
 - [8] Oman, P., Hagemeister, J.: Metrics for assessing a software system’s maintainability. In: Proceedings Conference on Software Maintenance 1992, pp. 337–338. IEEE Computer Society (1992)
 - [9] Pantiuchina, J., Lanza, M., Bavota, G.: Improving code: The (mis) perception of quality metrics. In: 2018 IEEE International conference on software maintenance and evolution (ICSME), pp. 80–91. IEEE (2018)
 - [10] PyCQA: Bandit (2025). URL <https://github.com/PyCQA/bandit>. Online; accessed 10 Jul 2025